

Git: Push & Branching Practice Lab — Questions

Q1. Number Utility Program

Write a Python file `numbers.py` with one function `is_even(n)` that returns `True/False`, plus a `__main__` block that tests it on the value 10. Run the file once in the terminal to confirm the output.

- Initialize a Git repository, check the file status, stage `numbers.py`, commit it with a meaningful message, rename the branch to `main`, connect it to a GitHub repository named `number-utils` as origin, and push with the upstream set.
- Create a branch `feature-square`, add a function `square(n)` to the file, and commit the change on that branch.
- Create a branch `feature-cube`, add a function `cube(n)` to the file, and commit the change on that branch.
- Switch back to `main`, merge both feature branches into it one after another, and push the updated `main` to GitHub.

Q2. Temperature Converter Program

Write a Python file `temperature.py` with a function `celsius_to_fahrenheit(c)` and a `__main__` block that converts the value 25. Run it once to confirm the output.

- Initialize the repository, run `git status` before and after staging the file, commit it, set the branch to `main`, add a remote origin pointing to a GitHub repository named `temp-converter`, and push with the upstream set.
- Create branch `feature-fahrenheit-to-celsius`, add a function `fahrenheit_to_celsius(f)`, and commit it on that branch.
- Create branch `feature-kelvin`, add a function `celsius_to_kelvin(c)`, and commit it on that branch.
- List all branches along with their last commit using `git branch -v`, then merge both feature branches into `main` in order, and push the final `main` to GitHub.

Q3. String Length Checker Program

Write a Python file `strlen.py` with a function `count_vowels(s)` and a `__main__` block that tests it on the string "programming". Run it once to confirm the output.

- Set your Git identity (name and email), initialize the repository, stage and commit the file, rename the branch to `main`, add origin for a GitHub repository named `string-checker`, and push.
- Create branch `feature-consonants`, add a function `count_consonants(s)`, and commit it on that branch.
- Create branch `feature-uppercase-count`, add a function `count_uppercase(s)`, and commit it on that branch.
- View the commit history with `git log --oneline` before merging, then merge both feature branches into `main` and push the result.

Q4. To-Do List Program

Write a Python file `todo.py` with a function `add_task(tasks, task)` that appends a task to a list, and a `__main__` block that adds "Submit assignment" to an empty list and prints it. Run it once to confirm the output.

- Initialize the repository, stage and commit the file, rename the branch to `main`, connect it to a GitHub repository named `todo-app` as origin, and push with the upstream set.
- Create branch `feature-remove-task`, add a function `remove_task(tasks, task)`, and commit it on that branch.

- c) Create branch feature-clear-tasks, add a function clear_tasks(tasks), and commit it on that branch.
- d) Switch to main, view a graph of both branches with git log --oneline --graph --all, merge both feature branches into main, and push the result.

Q5. Simple Bank Balance Program

Write a Python file bank.py with a function deposit(balance, amount) that returns the new balance, and a __main__ block that starts at 1000 and deposits 500. Run it once to confirm the output.

- a) Initialize the repository, stage and commit the file, rename the branch to main, add a remote for a GitHub repository named bank-balance, and push.
- b) Create branch feature-withdraw, add a function withdraw(balance, amount) with a check for insufficient funds, and commit it on that branch.
- c) Create branch feature-interest, add a function add_interest(balance, rate), and commit it on that branch.
- d) Merge feature-withdraw into main first, then merge feature-interest. If both branches had changed the same line and the second merge produced a conflict, what would you do in VS Code before completing the commit? Push the final result once resolved.

Q6. Simple Library Book Counter Program

Write a Python file library.py with a function total_books(shelves) that takes a list of book counts (one per shelf) and returns their sum, plus a __main__ block that tests it on [12, 8, 15]. Run it once to confirm the output.

- a) Initialize the repository, stage and commit the file, rename the branch to main, connect it to a GitHub repository named library-tracker as origin, and push with the upstream set.
- b) Create branch feature-average-books, add a function average_books(shelves) that returns the average books per shelf, and commit it on that branch.
- c) Create branch feature-max-shelf, add a function max_shelf(shelves) that returns the highest book count among the shelves, and commit it on that branch.
- d) Switch to main, merge both branches into main one after another, and push the final result to GitHub.
- e) Once both features are safely merged, delete both feature branches locally with git branch -d, and delete feature-max-shelf from GitHub as well. What command would you use instead of -d if Git refused to delete a branch, and why does Git block the normal delete in the first place?